IT VEDANT • DEVOPS FUNDAMENTALS • JENKINS MODULE

KONEXA

Enterprise DevOps Operations Platform

JENKINS CI/CD IMPLEMENTATION

A project report documenting the Jenkins-based Continuous Integration and Deployment environment built for Konexa — covering pipeline design, agents, GitHub webhook automation, and frontend deployment.

PREPARED BY

Essakimuthu Muthu

VERSION

Konexa v2.0

REPOSITORY

github.com/Essakimuthukonar/Konexa

OVERVIEW

Project at a Glance

Konexa is an ongoing Enterprise DevOps Operations Platform built module by module while learning DevOps tools and practices. Instead of creating a new project for every module, each completed module becomes a permanent part of the same platform. This report covers **Konexa v2.0**, the Jenkins CI/CD module.

Objective

Automate the build, validation, packaging and deployment of the Konexa frontend using Jenkins, triggered automatically by GitHub commits — removing the need to manually build and deploy after every code change.

Current Version
Konexa v2.0 — Jenkins CI/CD

GitHub Repository
github.com/Essakimuthukonar/Konexa

Jenkins Version
Jenkins 2.568.2

Branch Tracked
main

Technology Stack

Jenkins | GitHub Webhook | Jenkins Pipeline / Jenkinsfile | Node.js / npm | Next.js | PM2 | SSH / SCP | Ubuntu Linux | AWS EC2

SYSTEM DESIGN

Project Architecture

Four components make up the current Jenkins CI/CD environment: the Jenkins Controller, two Jenkins agents (frontend and backend), and the Konexa application server.

Backend Agent — Available for Future Backend Workloads

The backend agent is part of the Jenkins distributed infrastructure but is not currently part of the deployment flow above. It is reserved for backend/API pipelines in a future Konexa version.

INFRASTRUCTURE

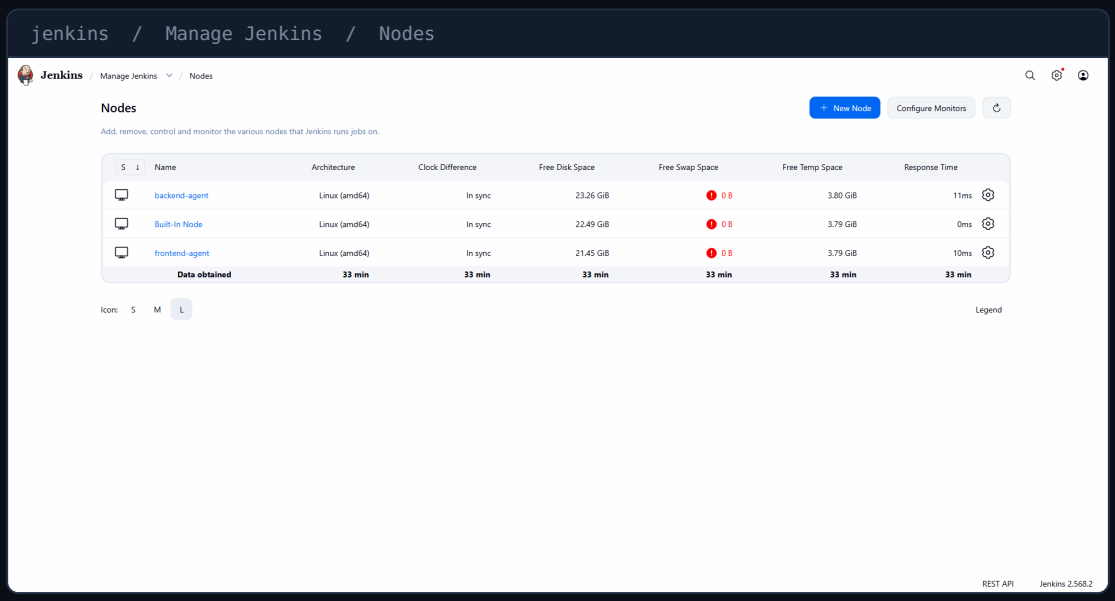
Jenkins Environment

The Jenkins environment is distributed across a controller and two agent nodes, each registered and monitored from the Jenkins **Nodes** panel.

Node	Role	Architecture
Jenkins Controller	Manages Jenkins, receives the GitHub webhook, schedules and coordinates builds	—
frontend-agent	Executes the Konexa frontend pipeline (build, test, package, deploy)	Linux (amd64)
backend-agent	Registered node for future backend workloads	Linux (amd64)
Konexa Server	Hosts the deployed Next.js frontend application via PM2	—

Why Use Jenkins Agents?

Agents let Jenkins run build work away from the controller itself, so the controller stays free to manage jobs while the actual clone/build/test/deploy work happens on a dedicated machine. Splitting frontend and backend agents also keeps future backend pipelines isolated from the frontend pipeline.



Screenshot: Jenkins Nodes panel showing **backend-agent**, **Built-In Node** and **frontend-agent**, all Linux (amd64) and reporting "In sync" clock status — confirming both agents are online and connected to the controller.

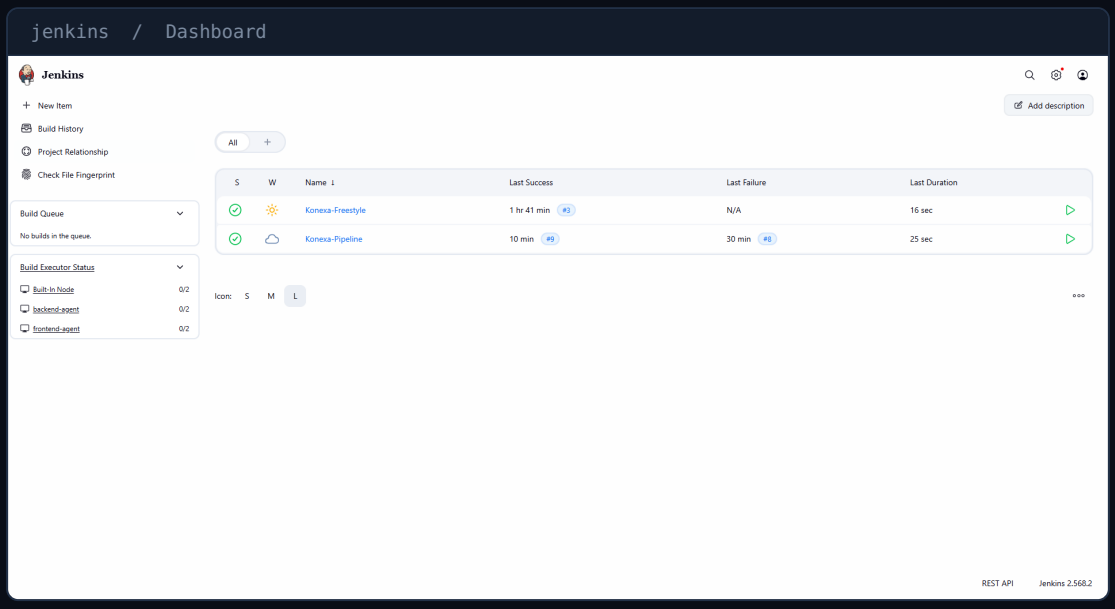
SETUP

Jenkins Installation & Configuration

Jenkins was installed and configured on an Ubuntu EC2 server to act as the CI/CD controller for the Konexa project.

Environment Prepared

- **Ubuntu Server** — base OS for the Jenkins Controller on AWS EC2
- **Java** — runtime required by Jenkins
- **Jenkins** — installed and configured as the CI/CD controller (version 2.568.2)
- **Git** — used by Jenkins to clone the Konexa repository
- **Node.js & npm** — required to install dependencies and build the Next.js frontend
- Administrative configuration of the Jenkins dashboard, nodes, and jobs



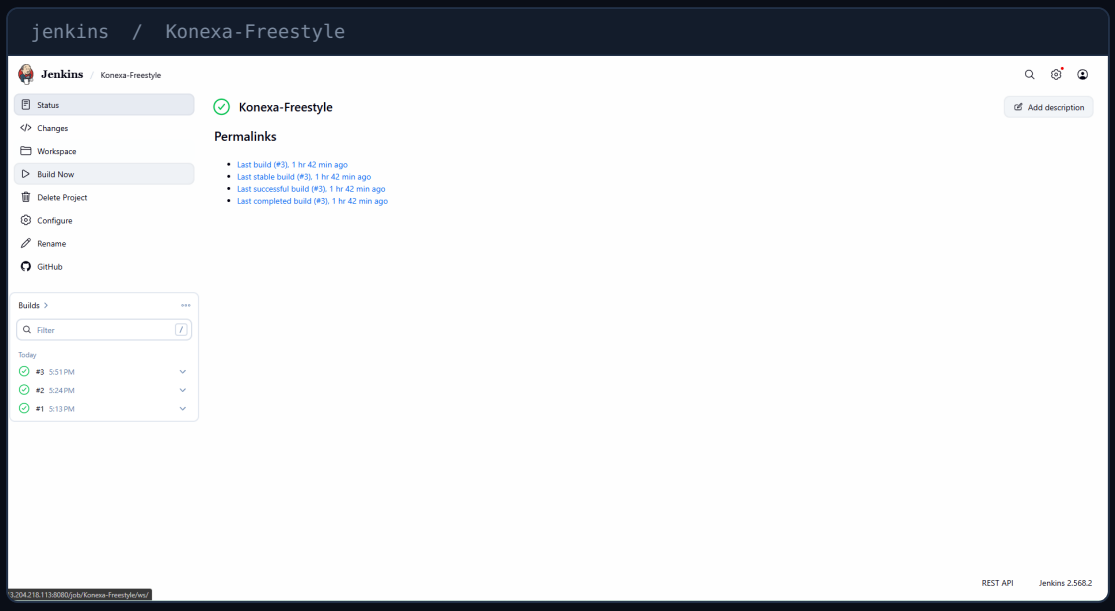
Screenshot: Jenkins dashboard showing the two configured jobs — **Konexa-Freestyle** and **Konexa-Pipeline** — along with the Build Executor Status panel listing the Built-In Node, backend-agent and frontend-agent.

Freestyle Job

Before building the full pipeline, a Jenkins **Freestyle project** was created as the first CI job for Konexa. This project type was used to learn the basics of build configuration, execution, and console output before moving to a Jenkinsfile-based pipeline.

Why a Freestyle Job First?

A Freestyle project is Jenkins' simplest job type, configured entirely through the UI. It was created and run to confirm that Jenkins was correctly installed, connected to the repository, and able to execute a build successfully before introducing pipeline-as-code complexity.



Screenshot: The **Konexa-Freestyle** job status page showing three completed builds (#1, #2, #3), all marked successful, with permalinks to the last build, last stable build, last successful build and last completed build.

PIPELINE AS CODE

Jenkins Pipeline

After the Freestyle job, the project moved to a **Jenkinsfile**-based pipeline named **Konexa-Pipeline**. The Jenkinsfile is committed to the Konexa GitHub repository and contains the working pipeline logic used by Jenkins.

Pipeline Stages

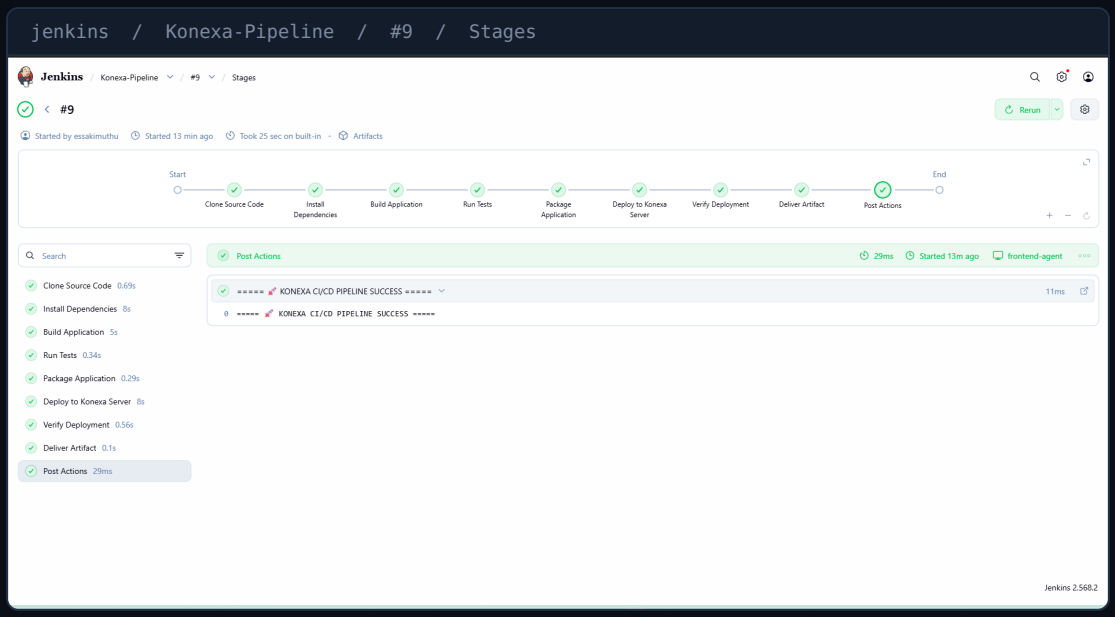
01 Clone Source Code	02 Install Dependencies	03 Build Application	04 Run Tests
05 Package Application	06 Deploy to Konexa Server	07 Verify Deployment	08 Deliver Artifact

Why Pipeline Over Freestyle?

A Jenkinsfile stores the entire build process as version-controlled code inside the repository, so the pipeline is repeatable, reviewable, and travels with the project. It also makes it possible to define multiple ordered stages — clone, install, build, test, package, deploy, verify, deliver — as a single visual, trackable workflow, instead of one flat build step.

EXECUTION

Pipeline Stage Execution



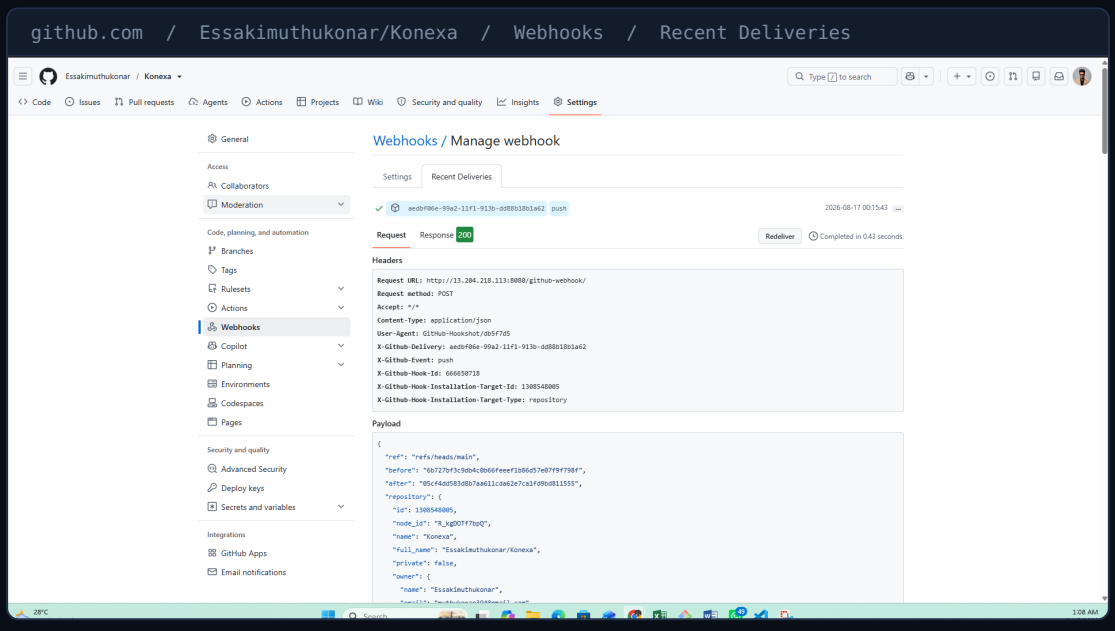
Screenshot: Build **#9** of Konexa-Pipeline, started by essakimuthu and completed in 25 seconds on the frontend-agent. All eight stages plus Post Actions are marked successful, and the console shows **"KONEXA CI/CD PIPELINE SUCCESS"**.

Stage	What It Does
Clone	Gets the latest code from the GitHub repository (main branch)
Install	Installs frontend dependencies using `npm ci`
Build	Creates the production Next.js build using `npm run build`
Test	Performs basic CI validation on the build output
Package	Creates the `konexa-frontend.tar.gz` artifact
Deploy	Copies the package to the Konexa server via SCP
Verify	Checks `pm2 status` and `curl localhost:3000`
Deliver	Archives the final artifact in Jenkins

AUTOMATION TRIGGER

GitHub Webhook & Continuous Integration

A GitHub webhook is configured on the Konexa repository so that every push to the **main** branch automatically triggers the Jenkins pipeline — no manual build step required.



Screenshot: A recent webhook delivery for a **push** event on `refs/heads/main`, sent to the Jenkins controller endpoint and answered with a **200** response, completed in 0.43 seconds — confirming the webhook is correctly wired to Jenkins.

CONTINUOUS DEPLOYMENT

Deployment

Once the frontend build is packaged, the Deploy stage ships it to the Konexa application server and brings it online using PM2.

Deployment Steps

- Copy the packaged frontend via **scp**
- Extract the package on the server
- Install production dependencies
- Remove the previous PM2 process
- Start the app with PM2 and save the process

Verification

```
pm2 status
curl -f http://localhost:3000
```

Confirms the process is running and the app responds on port 3000.

Note on Networking

Deployment uses the Konexa server's **private IP** (10.0.1.143) because Jenkins and the Konexa server communicate within the same AWS private network. The Jenkins Controller holds the Elastic IP (13.204.218.113); the Konexa server and frontend agent do not have Elastic IPs.

DATA SAFETY

Jenkins Backup

A Jenkins configuration and data backup was performed as part of the project, to preserve Jenkins' setup outside of the running instance.

Source

```
/var/lib/jenkins/
```

Backup Destination

```
/home/ubuntu/jenkins-backup
```

What Was Backed Up

- config.xml
- credentials.xml
- fingerprints
- github-plugin-configuration.xml

This is a straightforward Jenkins configuration/data backup — the core Jenkins configuration files and directories were copied to a separate location on the same server.

OBSERVABILITY

Pipeline Result & Monitoring

A pipeline report was captured from an earlier run (**Build #4**) documenting the build environment, stage results and build history.

Build	Trigger	Result
#1	Manual	FAILED — ESLint command unavailable
#2	Manual	SUCCESS
#3	Manual	SUCCESS
#4	GitHub Push Webhook	SUCCESS
#9	Manual / Webhook	SUCCESS — full 8-stage pipeline, 25 sec

Build Environment (Build #4)

- Jenkins 2.568.2
- Node.js v22.23.2
- npm 10.9.8
- Git 2.53.0
- Next.js 16.2.6

Issues Observed

- Multiple lockfiles detected by Next.js
- npm reported 11 vulnerabilities
- ESLint not available — basic CI validation used instead

Improvements Suggested

- Maintain a single package manager and lockfile
- Review and resolve npm dependency vulnerabilities
- Add proper automated unit tests
- Add ESLint as a project dependency for real lint validation
- Configure Jenkins build caching to reduce build time
- Use managed Jenkins credentials for private repository access

TROUBLESHOOTING

Challenges & Solutions

ESLint Not Available

Build #1 failed because the ESLint command was unavailable in the project dependencies. **Solution:** replaced the lint step with a basic CI validation check on the build output (package.json, .next, next.config.mjs) so later builds pass without ESLint.

Multiple Lockfiles Warning

Next.js detected more than one lockfile in the project during the build. **Solution:** noted as an improvement to standardize on a single package manager and lockfile going forward.

npm Dependency Vulnerabilities

npm reported 11 vulnerabilities during dependency installation. **Solution:** logged as a follow-up action item to review and patch the affected packages.

Private Network Deployment

The Konexa server has no public/Elastic IP for direct access. **Solution:** deployment communicates over the AWS private network using the server's private IP (10.0.1.143) instead.

REFLECTION

Learning Outcomes

Building Konexa v2.0 covered the full breadth of Jenkins fundamentals taught in this module:

Jenkins Administration
Installing and configuring Jenkins on Ubuntu, managing the dashboard and nodes.

Freestyle Jobs
Creating and running a Freestyle project as a first CI job.

Pipeline & Jenkinsfile
Writing a multi-stage pipeline as version-controlled code.

Jenkins Agents
Registering and using distributed frontend/backend agent nodes.

SSH Authentication
Using SSH/SCP for secure agent-to-server deployment.

GitHub Integration & Webhooks
Connecting GitHub to Jenkins for automatic pipeline triggers.

CI/CD Concepts
End-to-end flow from commit to a running application.

Artifact Management
Packaging and archiving build artifacts in Jenkins.

Deployment Automation
Automating package transfer, extraction and PM2 restart.

Jenkins Backup
Backing up Jenkins configuration and data for safekeeping.

SUMMARY DIAGRAM

Final CI/CD Flow

“ From Code Commit to Running Application ”

WRAP-UP

Conclusion

Konexa v2.0 adds a working Jenkins CI/CD layer on top of the AWS and Linux foundation built in v1.0. Code pushed to GitHub is now automatically cloned, built, tested, packaged, deployed and verified on the Konexa server — with no manual steps in between.

What's Implemented Today

Jenkins Controller + 2 agents · Freestyle job · 8-stage Jenkinsfile pipeline · GitHub webhook trigger · SCP/SSH deployment to the Konexa server · PM2 process management · deployment verification · Jenkins configuration backup.

What's Next

The next planned stages for Konexa are not yet implemented, but are the natural next steps after this CI/CD foundation:

Docker | Kubernetes | Terraform | Ansible | Monitoring

KONEXA PROJECT

Connect · Automate · Scale